# AI Lab assignment 14

Tejas Patil
U24AI061

Q1)

```python
def forward_chaining(rules, remaining_conditions, known_facts, goal):

    # List of facts to process (queue)
    to_process = list(known_facts)

    # Track processed facts to avoid repetition
    processed = {}

    while to_process:
        fact = to_process.pop(0)

        # Check if goal is reached
        if fact == goal:
            return True

        # Process only if not already used
        if fact not in processed:
            processed[fact] = True

            # Check all rules
            for rule_id in rules:
                conditions, result = rules[rule_id]

                # If current fact is part of rule conditions
                if fact in conditions:
                    remaining_conditions[rule_id] -= 1

                    # If all conditions satisfied → apply rule
                    if remaining_conditions[rule_id] == 0:
                        to_process.append(result)

    return False
```

```
rules_1a = {
    1: (['P'], 'Q'),
    2: (['L', 'M'], 'P'),
    3: (['A', 'B'], 'L')
}

remaining_conditions_1a = {
    1: 1,
    2: 2,
    3: 2
}

known_facts_1a = ['A', 'B', 'M']
goal_1a = 'Q'

result_1a = forward_chaining(
    rules_1a,
    remaining_conditions_1a.copy(),  # copy to avoid modification
    known_facts_1a,
    goal_1a
)

print("Test Case 1(a): Goal reached ->", result_1a)


rules_1b = {
    1: (['A'], 'B'),
    2: (['B'], 'C'),
    3: (['C'], 'D'),
    4: (['D', 'E'], 'F')
}

remaining_conditions_1b = {
    1: 1,
    2: 1,
    3: 1,
    4: 2
}

known_facts_1b = ['A', 'E']
```

```
goal_1b = 'F'

result_1b = forward_chaining(
    rules_1b,
    remaining_conditions_1b.copy(),
    known_facts_1b,
    goal_1b
)

print("Test Case 1(b): Goal reached ->", result_1b)
```

## Q2)

```
def backward_chaining(rules, known_facts, goal, visited):

    # If goal is already a known fact
    if goal in known_facts:
        return True

    # Avoid infinite loops
    if goal in visited:
        return False

    visited.add(goal)

    # Check all rules
    for conditions, result in rules:

        # If this rule can produce the goal
        if result == goal:

            # Check if ALL conditions can be proven
            all_true = True
            for condition in conditions:
                if not backward_chaining(rules, known_facts, condition,
visited):
                    all_true = False
                    break
```

```
            if all_true:
                return True

    return False


rules_2a = [
    (['P'], 'Q'),
    (['R'], 'Q'),
    (['A'], 'P'),
    (['B'], 'R')
]

known_facts_2a = ['A', 'B']
goal_2a = 'Q'

result_2a = backward_chaining(
    rules_2a,
    known_facts_2a,
    goal_2a,
    set()
)

print("Test Case 2(a): Goal proven ->", result_2a)


rules_2b = [
    (['A'], 'B'),
    (['B', 'C'], 'D'),
    (['E'], 'C')
]

known_facts_2b = ['A', 'E']
goal_2b = 'D'

result_2b = backward_chaining(
    rules_2b,
    known_facts_2b,
    goal_2b,
    set()
```

```
)

print("Test Case 2(b): Goal proven ->", result_2b)
```